CHARUSAT
Devang Patel Institute of Advance Technology and Research

Web Development Framework (ITUE203)

HTTPS IS NOT ENOUGH

How Laravel Protects Passwords, Cookies, Sessions and User Data

Laravel

Secure Build Better Web Apps

1 THE PROBLEM
Why is HTTPS not enough?

HTTPS protects data while it travels between your browser and the server, but it cannot protect against:

- Passwords stored in plain text.
- Session cookies being misused.
- Unauthorized access to user data.
- Forged requests (CSRF attacks).

We need additional application-level security provided by a web framework like Laravel.

2 OUR SOLUTION
Multiple Security Layers

Laravel provides a set of powerful features to protect your web application:

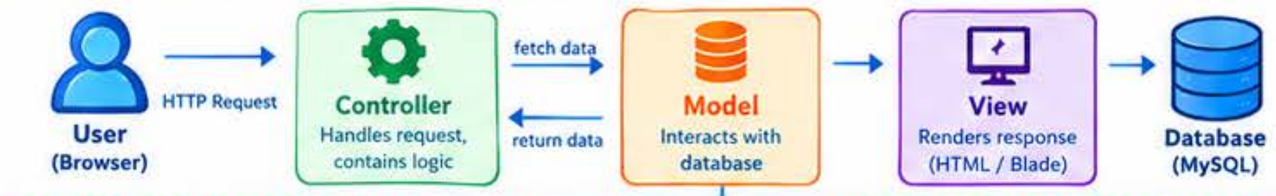
- **HTTPS/TLS** — Protects data while it travels.
- **Password Hashing** — Stores a password hash, not the original password.
- **Cookies** — Carries session ID in browser.
- **Sessions** — Maintains the user's login state.
- **Authentication** — Verifies who the user is.
- **Authorization** — Checks what the user can access.
- **CSRF Protection** — Prevents forged requests using tokens.

Key idea: Each layer protects a different type of risk. Together they provide complete security.

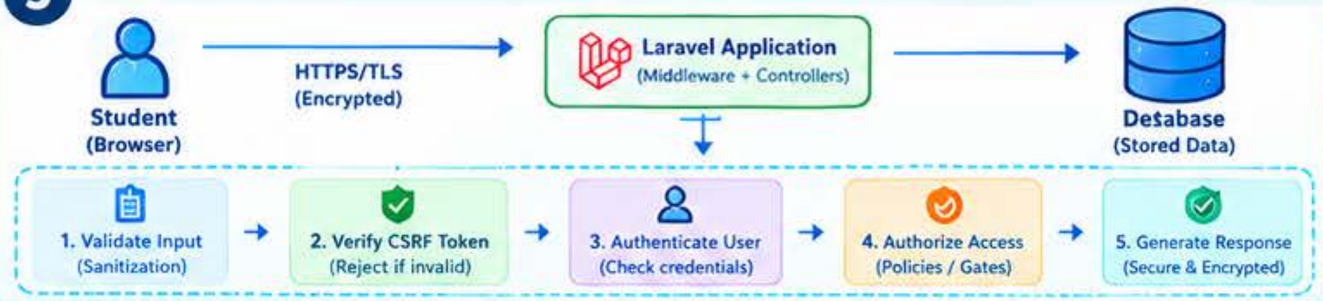
3 HOW DOES IT WORK?
Login to Secure Access — Step by Step

1. **Enter Credentials** — Student enters email & password over HTTPS.
2. **HTTPS/TLS** — Encrypts data during transfer.
3. **Password Hashing** — Compares input with stored hash (bcrypt/Argon2).
4. **Session** — Creates/updates session (cookie).
5. **Authentication** — Verifies logged-in user.
6. **Authorization** — Checks access to requested resource.
7. **Response** — Secure data returned over HTTPS.

4 LARAVEL ARCHITECTURE (MVC)
A structured framework for building web applications



5 SECURITY FLOW DIAGRAM (with CSRF Protection)



6 PASSWORD HASHING
Secure storage, not encryption

- Uses **bcrypt** or **Argon2** (one-way hash).
- Original password cannot be retrieved.
- Verified during login, not decrypted.

```php
use Illuminate\Support\Facades\Hash;
// Store hashed password
$hashed = Hash::make($password);
// Verify during login
if (Hash::check($input, $hashed)) {
    // Login successful
}
```

Remember: Hashing protects stored passwords; HTTPS protects data in transit.

7 BENEFITS

- Better protection for sensitive information.
- Secure password storage.
- Prevents session hijacking.
- Controls access with authentication & authorization.
- Reduces CSRF and other common attacks.
- Builds reliable and scalable web applications.

8 LIMITATIONS & CHALLENGES

- Weak passwords can still be guessed.
- Session cookies can create risks if stolen.
- Incorrect permission checks can expose data.
- HTTPS cannot fix every vulnerability.
- Laravel features must be configured correctly.

9 REAL-WORLD EXAMPLE — STUDENT HUB PORTAL



How Laravel secures it:

- HTTPS protects login communication.
- Hashing protects stored passwords.
- Sessions maintain login state.
- Authentication verifies the student.
- Authorization restricts access to your own data.
- CSRF protection prevents forged requests.

10 DEVELOPMENT & DEPLOYMENT WORKFLOW



11 CONCLUSION

- HTTPS is important, but not enough.
- Laravel provides multiple layers of security.
- Together they help build safer, organized and reliable web applications.

"Security is not a feature, it's a process."

Student Name:
Bhatti Meet - 25DCE010

Aryan Chauhan - 25DCE016